Supplementary Information

S1 General Ising Model with b $\neq$ 0

Given a spin vector $\mathbf{s} = [s_1, s_2, .., s_D]$ and an external bias vector (or a field) $\mathbf{b}$, a general Ising Hamiltonian has the form:

$$\min_{\mathbf{s}\in\{-1,+1\}^D} H(\mathbf{s}) = \frac{1}{2}\mathbf{s}^\intercal\mathbf{Q}\mathbf{s} + \mathbf{b}^T\mathbf{s} \tag{1}$$

Then, following the steps in the Methods section 4 leads to

$$\Delta H_n = 2\sum_{p\in\mathcal{C}} \Delta s_{p,n}\left[\left(\sum_{j=1}^{D} Q_{pj}s_{j,n}\right) + b_p\right], \tag{2}$$

where the set $\mathcal{C} = \{i : \Delta s_{i,n} \neq 0\}$. Introducing a non-spiking static neuron whose state $s_0 = 1$ remains constant, then Eq. 2 can be written as

$$\Delta H_n = 2\sum_{p\in\mathcal{C}} \Delta s_{p,n}\sum_{j=0}^{D} Q_{pj}s_{j,n}, \tag{3}$$

which has the same form as Eq. 8 but with $Q_{p,0} = b_p$. Alternatively, the external field b_p for neuron p can be interpreted as a shift in the membrane potential which can be programmed when initializing the neurons.

S2 Mapping of MAX-CUT to Ising Model

Consider a generic MAX-CUT problem on graph, $G = \{V, \mathbf{W}\}$, where $V = \{v_i | i \in 1...D\}$ denotes the set of D vertices and $\mathbf{W} \in \{-1, 0, 1\}^{D\times D}$ denote the weighted adjacency matrix. Each of the vertices v_i is connected to any other vertex v_j through connection W_{ij}. A cut on G partitions the set of vertices V into sets L and R. The vertices belong to different sets can be described by an additional variable s_i associated with each node v_i, following

$$s_i = \begin{cases} +1 & v_i \in L \\ -1 & v_i \in R. \end{cases} \tag{4}$$

The edge weight W_{ij} connecting v_i and v_j is cut only when $s_i s_j = -1$. MAX-CUT problem aims to maximize the number of cuts which is given by

$$\max_{s_i,s_j\in\{-1,+1\}} H_{max} = \frac{1}{2}\sum_{i,j}^{D} W_{ij}(1 - s_i s_j). \tag{5}$$

Maximizing the total number of cuts is therefore equivalent to minimizing the following Ising energy function with the Hamiltonian matrix $\mathbf{Q}$ equaling to the weighted adjacency matrix $\mathbf{W}$,

$$\max_{\mathbf{s}\in\{-1,+1\}^D} H_{Ising} = \frac{1}{2}\mathbf{s}^\intercal \mathbf{Q}\mathbf{s}. \tag{6}$$

S3 Mapping of MAX-INDEPENDENT SET to Ising Model

The Max-Independent Set (MIS) problem on a graph $G = \{V, \mathbf{W}\}$ can also be mapped to an Ising model. In this context, $V = \{v_i | i \in 1 \dots D\}$ represents the set of D vertices, and $\mathbf{W} \in \{0,1\}^{D\times D}$ denotes the unweighted adjacency matrix, where $W_{ij} = 1$ if there is an edge between vertex v_i and vertex v_j, and $W_{ij} = 0$ otherwise. In the MIS problem, the goal is to find the largest subset of vertices $T \subseteq V$ such that no two vertices in T are adjacent in $\mathbf{W}$. An additional variable x_i can be associated to any node v_i, such that it represents if the node is in subset T as

$$x_i = \begin{cases} +1 & v_i \in T \\ 0 & v_i \notin T. \end{cases} \tag{7}$$

while the independent condition is satisfied such that $\forall\, v_i, v_j \in T$, $W_{ij} = 0$.

Therefore to enforce the aforementioned independent constraint and maximizing the size of the independent set, the objective function can be formulated as

$$\max_{x_i,x_j\in\{0,+1\}} H_{mis} = \left(\sum_i^D x_i x_i\right) - \beta\left(\sum_{i,j}^D W_{ij} x_i x_j\right), \tag{8}$$

where $\beta \in (0,1)$ is a hyperparameter for applying penalty on solutions that violate the constraint. The Ising Hamiltonian $\mathbf{Q}$ can be formulated as $Q_{ij} = W_{ij}, Q_{ii} = -1, \forall i,j \in 1...D$. Therefore, maximizing the independent set is equivalent to minimizing the Ising energy in Eq. 6, where the variable $\mathbf{x}$ can be mapped to spin states $\mathbf{s}$ following $\mathbf{s} = 2\mathbf{x} - 1$. Different from the formulation presented in the Methods Section 4.1, since $Q_{ii} \neq 0$ in the MIS formulation, the pseudo-gradient changes as the following,

$$\Delta H_n = 2\left(\Delta s_{p,n}\left[\sum_{j=1}^D Q_{pj} s_{j,n-1}\right] + Q_{pp}\right), \tag{9}$$

where we have used $\Delta s_{p,n}^2 = 1$. Hence, $\Delta H_n < 0$, if and only if

$$\Delta s_{p,n}\left[\sum_{j=1}^D Q_{pj} s_{j,n-1}\right] + Q_{pp} < 0. \tag{10}$$

S4 Algorithmic Implementation of NeuroSA

The NeuroSA architecture is simulated on a CPU platform using the MATLAB R2022a software package. The ON-OFF neuron pair parameters, $\mathbf{v}^{\pm}$, $\mathbf{s}^{\pm}$, and $\Delta\mathbf{s}^{\pm}$ are stored in pre-allocated arrays. As described in Methods section 4.4, the Bernoulli random variable $\mathcal{N}_n^B$ can be decoupled from the firing threshold μ_n such that the threshold for each neuron pair only emulates the simulated annealing acceptance/rejection dynamics while the ergodicity is enforced by using a global random selection arbiter. The software simulation follows this implementation by generating an array of i.i.d random numbers and calculating the decision threshold accordingly. The pseudo-code for the NeuroSA software is presented as the following

Algorithm 1 NeuroSA Pseudo-code

$\mathbf{s}^+ \leftarrow 1$, $\mathbf{s}^- \leftarrow 0$ ▷ Spin states initialization
$\Delta\mathbf{s}^+ \leftarrow 0$, $\Delta\mathbf{s}^- \leftarrow 0$ ▷ Spikes initialization
$\mathbf{v}^+ \leftarrow \mathbf{Q}(\mathbf{s}^+ - \mathbf{s}^-)$, $\mathbf{v}^- \leftarrow -\mathbf{Q}(\mathbf{s}^+ - \mathbf{s}^-)$ ▷ Membrane potential initialization
$iter \leftarrow 1$
$t \leftarrow 1$
while $iter <$ MAX_ITER **do**
  $\mathbf{thld} \leftarrow d * \left(\frac{\beta \log U(0,1)}{1+\log(-\alpha t)}\right)$ ▷ Distinct threshold for each neuron pair
  for $i = 1$ to D **do**
    if $\mathbf{s}^+[i] = 0$ and $\mathbf{v}^+[i] > \mathbf{thld}[i]$ **then** ▷ ON neuron firing criteria
      $\Delta\mathbf{s}^+[i] \leftarrow 1$
      $\Delta\mathbf{s}^-[i] \leftarrow 0$
    else if $\mathbf{s}^-[i] = 0$ and $\mathbf{v}^-[i] > \mathbf{thld}[i]$ **then** ▷ OFF neuron firing criteria
      $\Delta\mathbf{s}^+[i] \leftarrow 0$
      $\Delta\mathbf{s}^-[i] \leftarrow 1$
    else
      $\Delta\mathbf{s}^+[i] \leftarrow 0$
      $\Delta\mathbf{s}^-[i] \leftarrow 0$
    end if
  end for
  randomly select neuron p from $(\Delta\mathbf{s}^+ - \Delta\mathbf{s}^-) \neq 0$ ▷ Inhibitive firing
  $\mathbf{s}^+[p] \leftarrow \mathbf{s}^+[p] + \Delta\mathbf{s}^+[p] - \Delta\mathbf{s}^-[p]$
  $\mathbf{s}^-[p] \leftarrow \mathbf{s}^-[p] - \Delta\mathbf{s}^+[p] + \Delta\mathbf{s}^-[p]$
  $\mathbf{v}^+ \leftarrow \mathbf{v}^+ + 2\mathbf{Q}[p,:]\Delta\mathbf{s}^+ - 2\mathbf{Q}[p,:]\Delta\mathbf{s}^-$
  $\mathbf{v}^- \leftarrow \mathbf{v}^- - 2\mathbf{Q}[p,:]\Delta\mathbf{s}^+ + 2\mathbf{Q}[p,:]\Delta\mathbf{s}^-$
  $t \leftarrow t + dt$
end while

Here, MAX_ITER denotes the maximum simulation time in discrete steps, dt is the granularity of the time step, and α, β are the hardware-related hyperparameter of the FN annealer as discussed in Methods section 4.3. This implementation faithfully recovers the asynchronous NeuroSA architecture in that it instantiates distinctive noisy

thresholds for every individual neuron pair. However, for large-scale implementation, the simulation runtime is determined by the random generation function [1]. Therefore, we implemented a more lightweight software for large-scale simulation that reduces the footprint for random number generation. The neurons are randomly marked for selection, regardless of their firing status (active neurons). Then based on the threshold μ_n, the selected neuron could fire based on the spiking criterion. This implementation is different from the synchronous implementation Algorithm 1 because only one noisy threshold is generated at a given time-instant, which reduces the CPU runtime.

S5 Mapping NeuroSA on SpiNNaker2

SpiNNaker2 is a MultiProcessor System on Chip (MPSoC) in 22nm FDSOI technology designed to execute event-based machine learning, neuromorphic, and hybrid models [2]. SpiNNaker2-based systems are intended to be scaled up from one standalone chip composed by 152 Arm-based Processing Elements (PEs), to supercomputer levels with millions of PEs interconnected in a Globally Asynchronous Locally Synchronous configuration. A single chip (Fig. S1b) contains 38 Quad-core Processing Elements, each of which employs four Arm-based PEs with custom accelerators. All the resources within a single chip are interconnected via a light-weight Network-on-Chip (NoC) as in Fig. S1c, and operate under an interrupt-driven approach for dynamically managing power consumption. As a neuromorphic system, SpiNNaker2 implements neuron models via precompiled software that is executed in the Arm-based PEs, and uses its native communication infrastructure to redirect the spike-based activity across the system. A SpiNNaker router within each chip is the responsible component to extend the interdependent hierarchies to multi-chip levels, and beyond that to multi-board, multi-frames, and multi-rack levels. The overall topology of SpiNNaker2-based systems employs a toroidal mesh with each chip connecting to six neighbors via a predefined configuration that ensures the short communication delays within the system. SpiNNaker2 is among the most flexible neuromorphic chips providing customizations in both communication and computation to deploy more than 10 billion neurons (i.e., more than 1,000 per PE) and beyond 10,000 synapses (i.e., more than 1 million per core) in a single system [3].

The neuron model in NeuroSA is implemented through embedded software on SpiNNaker2 cores, while the high-level control and experiment configuration are performed through py-spinnaker2 [4], a Python library and high-level API designed for programming SpiNNaker2. Given an input MAX-CUT graph, we construct a NeuroSA network, where a mapper module within py-spinnaker2 determines the number of cores used as well as the distribution of neurons per core. This mapping depends on the network's size, as measured by the number of neurons and synaptic connections. Following the architecture of the synchronous NeuroSA, as outlined in Methods section 4.4, we designed a global arbiter to perform the outer-loop level random selection across all active neurons at any time step. The arbiter uniformly samples one core ID from all used core IDs per time step. This selected core is the only core that is allowed to emit a spike at that time step. Following core selection, we update the membrane potentials for all neurons. Among neurons whose membrane potentials crossed

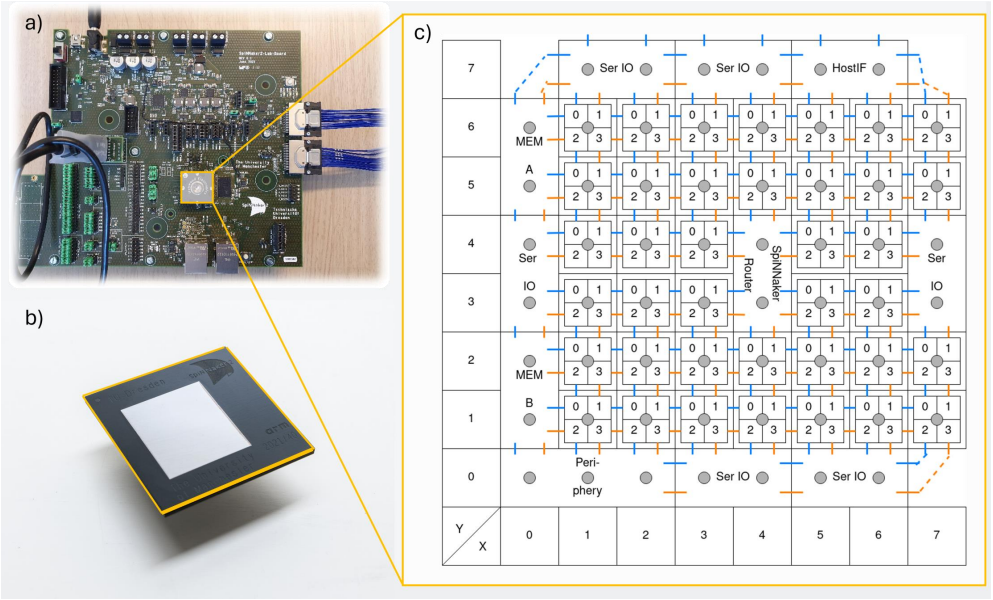


Fig. S1 (a) Testing board used for the NeuroSA experiments, (b) highlighting its single SpiNNaker2 chip (c) and its internal topology.

the threshold on the selected core, the global arbiter uniformly samples only one neuron to spike. If none of the neurons on the selected core crossed the threshold, no spike is emitted at that time step. The uniform sampling of the cores as well as neurons that are selected for spike emission is done using the on-chip true random number generator [5]. Fig. S2 depicts the firing dynamics of the NeuroSA implementation on SpiNNaker2.

S5.1 Bare metal Implementation on SpiNNaker2

We developed a bare metal NeuroSA implementation on the ARM micro-processors that bypassed the SpiNNaker2 software stack. As a result, the run time is significantly optimized comparing to the SpiNNaker2 software implementation($\sim 40\times$).

The NeuroSA neuron is compiled directly into micro-code that gets executed by the ARM PE cores, without going through the hierarchical general-purpose software stack. Given an input graph, neurons (nodes) are allocated to cores while ensuring that their memory requirements remain within the 128 KB limit of the local PE SRAM, taking the graph's density into account. For denser graphs, fewer neurons can be stored on a single PE due to the increased size of the routing table required to store fan-out neuron indices. The mapping is done in a static fashion where little load balancing is performed across cores. Since the communication overhead within cores are negligible and the inter-core communication creates bottleneck for transmission delay, the static mapping may account for the delay in execution in the current implementation.

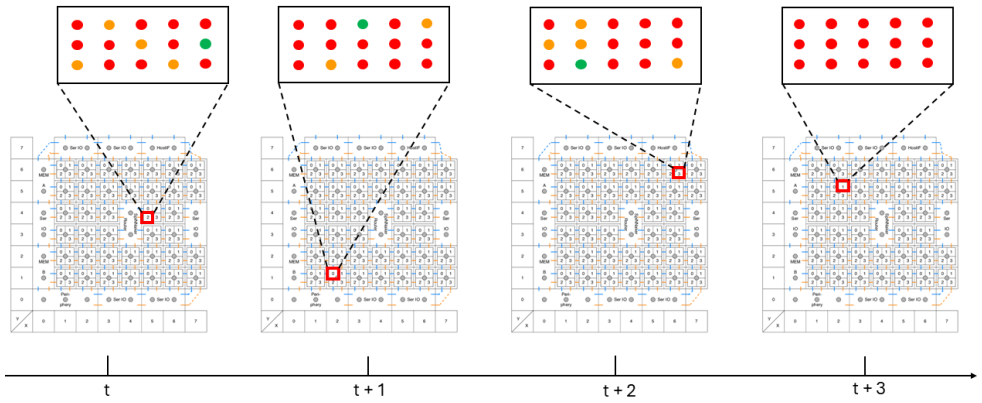


Fig. S2 Overview of the random sampling of cores and neurons on SpiNNaker2. Each time step displays the internal topology of SpiNNaker2, with red squares indicating the selected core emitting a spike at that time step. Zooming in on the chosen reveals its neurons: those in red have not crossed the threshold, while those in yellow have. The green neuron signifies the spike emitter. At the final time step (t+3), no neurons have crossed the threshold, resulting in no spike emission.

Alongside the neuron cores, one core is designated as an orchestrator. The orchestrator PE samples a random neuron at each time step using the on-chip TRNG. It then sends a core-to-core SpiNNaker packet to the corresponding core, including the selected neuron's index in the payload, to trigger computation. Each of the neurons instantiated on a PE stores the fan-in address table to the RAM that is local to the core. This design simplifies the inter-core communication protocol: the asynchronous spikes are transmitted to the post-synaptic neurons sequentially the receiving neurons are responsible for checking and re-weighting the incoming spike and update their membrane potentials respectively. However, the sequential transmission spikes in the current implementation undermines the in-built parallelism on neuron state updates provided by the NeuroSA architecture. The lack of multi-casting ability may also be one of the reasons for the execution time of the current implementation.

Ongoing and future efforts will focus on identifying optimal strategies for mapping neurons onto cores, tailored to the size and density of the input graph, to minimize execution time by balancing the computation-communication tradeoff. Furthermore, this work could be enhanced by leveraging the SpiNNaker 2 communication infrastructure, such as utilizing multicast packets to broadcast information to multiple cores simultaneously.

S5.2 Neuromorphic Advantage on SpiNNaker2

We compare the time-to-solution for both CPU and SpiNNaker2 platforms when NeuroSA is run for the same number of execution cycles. The result is shown in Fig. S3**a**. The CPU platform comprises of an Intel® Core Ultra 9 185H processor core running at a maximum clock rate of 2.5 GHz whereas the SpiNNaker2 ARM core running at a clock rate of 300MHz. The number of iterations for NeuroSA execution is fixed across

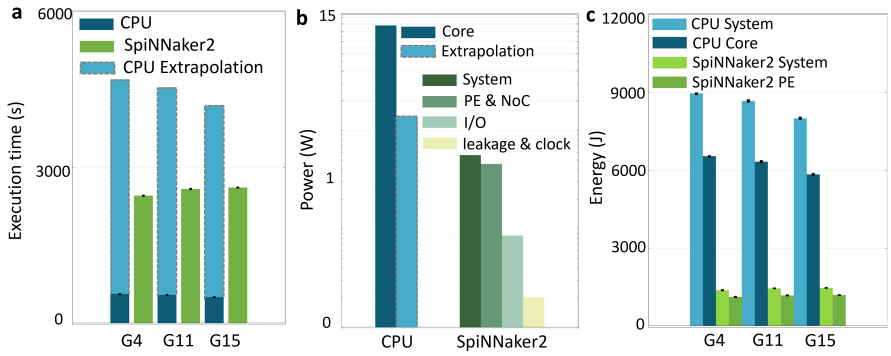


Fig. S3 Presents the timing and energy metrics on both CPU and SpiNNaker2 systems that executes the same-sized NeuroSA MAX-CUT workloads (800 nodes). (a) Plots the real time-to-solution in seconds for both systems and extrapolated CPU time with respect to SpiNNaker2 clock frequency. (b) Reports the instantaneous power consumption and breakdown of it for SpiNNaker2 and CPU system. (c) Plots the energy-to-solution for both platforms, where the error bars show the maximum and minimum energy dissipation across runs.

multiple runs on both the platforms and for different graphs. In addition to reporting the the time-to-solution for both platforms, to ensure fair comparison, we linearly extrapolated the CPU time-to-solution to account for the difference between the CPU and SpiNNaker2 clock frequencies. The result is shown in Fig. S3**a**. The extrapolation is also justified because the NeuroSA algorithm searches the optimization landscape sequentially as the annealing schedule changes over time. The linear extrapolation serves as an optimistic lower bound on the CPU run time with lower clock frequency since it ignores the additional cascading delay that arises in the CPU pipeline due to the combination of same-speed memory and slower-clock CPU. Although the current implementation on SpiNNaker2 surpasses the CPU in terms of absolute run time due to network communication bottleneck and the sequential transmission of spiking events, the extrapolated CPU run time at equal clock frequency is worse than the SpiNNaker2 execution time.

Furthermore, we measured the NeuroSA power consumption on both CPU and SpiNNaker2 when solving the same benchmark COP. For the CPU system, we report the system and the CPU core power consumption separately. We used HWiNFO® and Intel SOC Watch along with the Vtune Profiler [6] for the power measurement. While executing the NeuroSA algorithm, the CPU system consumes 15.93W, in which the CPU core takes up 11.63W. We perform similar linear extrapolation on the CPU core power consumption with respect to the SpiNNaker2 clock frequency. The extrapolation on CPU core power consumption indicate that such a core running at 300MHz would consume 1.39W. On the other hand, SpiNNaker2 NeuroSA implementation consumes 561.9mW. The built-in hardware sensors measures the breakdown of the power consumption as the following: the PEs (Processing Elements) and the Network on Chip (NoC) consumes 455.95mW, the I/O 85.7mW, and the rest 20.26mW that accounts for unused DRAM leakage and clock generation. The power analysis is

shown in Fig. S3**b**, where SpiNNaker2 demonstrate clear power advantage even to the extrapolated results.

Next, we show the energy-to-solution results for both platforms without performing extrapolation. As shown in Fig. S3**c**, for three different 800-node MAX-CUT problem, we plot the energy-to-finish for both platforms on the scale of entire system and comupting elements. As shown in the figure, for either the system or the processing unit, SpiNNaker2 triumphs CPU in terms of energy for completing the same NeuroSA workloads. Despite some non-optimality in the current SpiNNaker2 implementation of NeuroSA, as described in Supplementary Section S5.1, power-efficient neuromorphic hardware already exhibits certain energy benefits in executing the NeuroSA algorithm.

S6 Effect of finite precision on NeuroSA

The evolution of the threshold μ is the key to the robustness of the NeuroSA architecture. While CPU (or software) implementation can use floating-point precision, in practice many neuromorphic hardware accelerators support only finite precision arithmetic. The long-term vision is to deploy NeuroSA on custom-ASIC or a hardware platform to achieve high energy-efficiency and low time-to-solution. One option is to implement the neuron and network model using standard neuromorphic architectures such as Loihi or SpiNNaker2, where as the FN annealers are realized in analog or using mixed-signal approaches using an analog-to-digital or digital-to-analog converters. Here we explore how the NeuroSA performance degrades when the precision of the computation is reduced. The state of the neurons are all binary variables taking values in $\{-1, +1\}$, where the weights (or connectivity) are also quantized. Therefore, the integrate-and-fire dynamics produces membrane potentials that are also discrete integers, however, their range and precision are limited by the network size and fan-out. Therefore, the only component in the architecture that is affected by quantization is the firing threshold for each neuron. We applied quantization to the thresholding function before determining the spiking activity of a particular neuron and plotted the obtained solution for each precision.

As shown in Fig. S4, when quantized to 64- and 32-bit floating point, the noisy thresholds incurs identical neuron population dynamics, resulting in exactly the same evolution of the obtained results. When the precision is decreased to 16-bit the network dynamics differ from the high-precision cases but the overall performance in terms of solving the original optimization problem is similar, approaching the SOTA solution. However, when the precision is further decreased to 8-bit, the performance drops at the low-temperature region, as shown in Fig. S4 the purple curve. This result is as what we expected since the random fluctuation on the noisy threshold is vital to the performance of the NeuroSA architecture. When the temperature cools down, the amplitude of this fluctuation is also annealed. Under a low-precision scenario, the effect of the firing threshold fluctuation is concealed by the quantization effect. Therefore, the overall NeuroSA dynamics fail to follow the optimal simulated annealing dynamics which results in worse performance than the higher precision implementation.

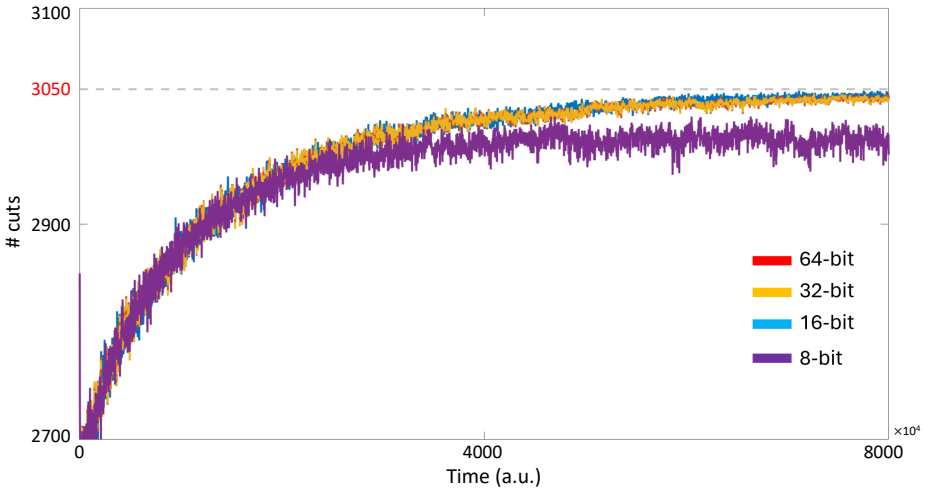


Fig. S4 Effect of finite precision on the performance of NeuroSA.

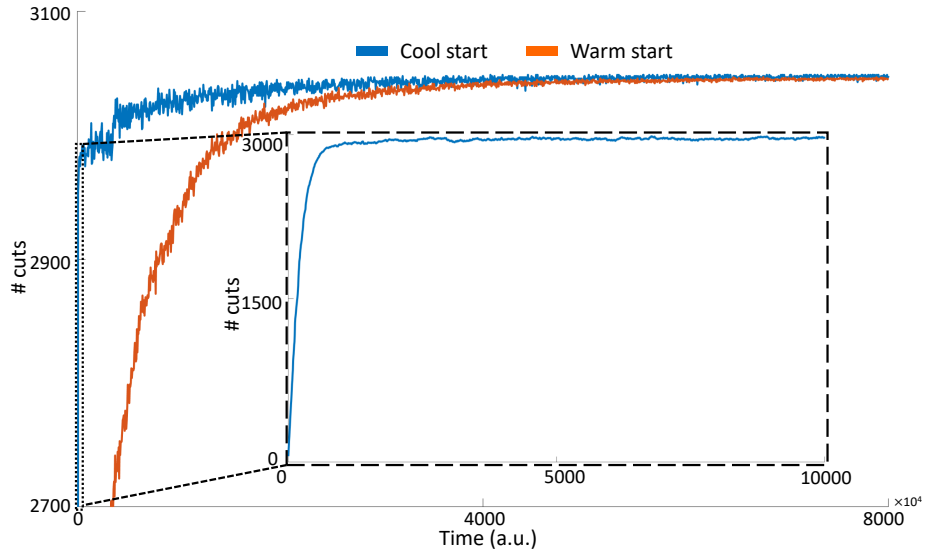


Fig. S5 Using a low-temperature start to converge fast to a solution neighborhood, after which the annealing is used for exploring the solution space and asymptotic convergence to SOTA solution

S7 NeuroSA Low-temperature Start

From Eq. 2 and 13, if NeuroSA is configured with a low-temperature T_n close to 0, the update of neuron states is determined by the network gradient, which results in faster convergence. This is because during the initial stages of the dynamics, any stochastic factors slows down the convergence. As shown in Fig. S5, the cold-start condition pushes the network to convergence to ~ 3000 cuts for the $G15$ benchmark for which

the SOTA is 3050. However, the dynamics stalls after $\sim 10^4$ iterations because the network is trapped in the neighborhood of a local attractor state. However, by re-heating (or adding noise to the threshold) the solution can be further improved, as shown in Fig. S5. As indicated in Fig.3e, the time to obtain a unit gain in solution takes up most of the entire duration, at the end of convergence. Therefore, the cold-start strategy accelerates only the initial phase of the optimization, and is intended for practical implementation when there is a simulation time constraint.

S7.1 Graph Maximum Fanouts/Latency

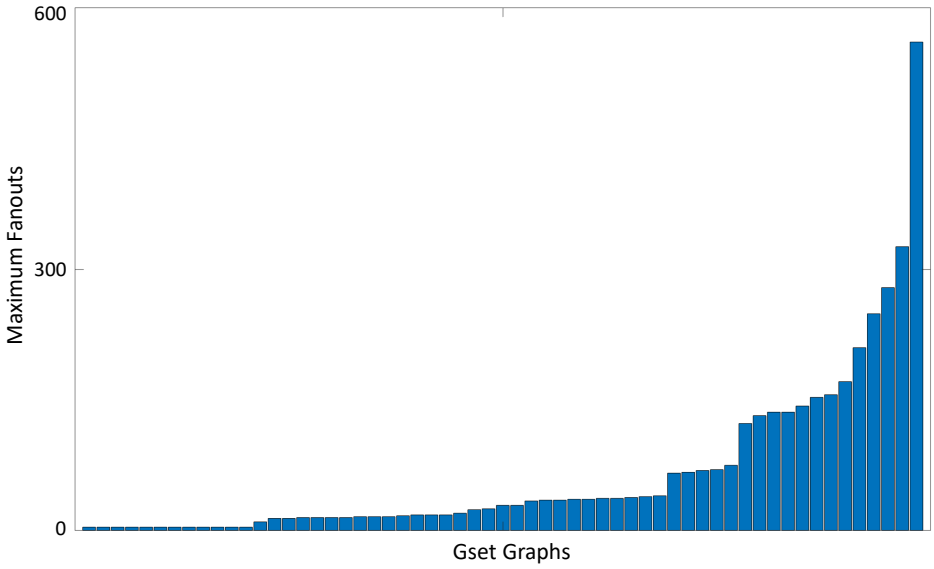


Fig. S6 Maximum fanouts on each of the Gset benchmarks

The latency for propagating spiking events in a conventional sequential implementation of the NeuroSA architecture is determined by the latency between the spike generation to the time when all target neurons receive the spike and estimate the pseudo-gradient. Because of the simplicity of our neuron model design, the delay is mainly embedded in the sequential transmission of the spiking event, due to the shared bus in the system. Therefore, the maximum latency is determined by the largest fanout in the problem graph. As indicated in Fig. S6, the maximum fanout among all tested Gset graphs is around 600, indicating a consecutive 600 incidents of routing the spiking events across the system interconnect. On the other hand, in neuromorphic hardware, where the neurons are implemented in parallel and distributed across the network, the transmission delay is offset by the large fanout of the physical interconnects between neuron pairs. Therefore, the neuromorphic implementation is not limited by fanout of the graph.

S8 Table of Comparison for Gset Benchmarks

The tables S1 and S2 summarize the SOTA solution reported for different Gset benchmarks [7] and the difference from the worst-case solution obtained by NeuroSA.

Table S1 Gset1-30 Results

Gset Benchmarks	SOTA Solution	NeuroSA
G1	11624	0
G2	11620	-3
G3	11622	0
G4	11646	-5
G5	11631	0
G6	2178	0
G7	2006	0
G10	2000	-1
G11	564	0
G12	556	0
G13	582	0
G14	3064	-1
G15	3050	-1
G16	3052	0
G17	3047	-2
G18	992	-4
G19	906	-1
G20	941	0
G21	931	-3
G22	13359	-1
G23	13344	-3
G24	13337	-2
G25	13340	-7
G26	13328	-4
G27	3341	0
G28	3298	-2
G29	3405	-14
G30	3413	-1

Table S2 Gset 31-59, 67, 72 Results

Gset Benchmarks	SOTA Solution	NeuroSA
G31	3310	-2
G32	1410	-4
G33	1382	-2
G34	1384	-2
G35	7687	-12
G36	7680	-17
G37	7691	-8
G38	7688	-16
G39	2408	-3
G40	2400	-7
G41	2405	-12
G42	2481	-15
G43	6660	0
G44	6650	0
G45	6654	0
G46	6649	-3
G47	6657	-1
G48	6000	0
G49	6000	0
G50	5880	0
G51	3848	-1
G52	3851	-4
G53	3850	0
G54	3852	-4
G55	10299	-15
G56	4017	-11
G57	3494	-22
G58	19293	-39
G59	6086	-31
G67	6950	-68
G72	7006	-76

References

[1] Pierog, T., Karpenko, I., Katzy, J.M., Yatsenko, E., Werner, K.: Epos lhc: Test of collective hadronization with data measured at the cern large hadron collider. Phys. Rev. C **92**, 034906 (2015) https://doi.org/10.1103/PhysRevC.92.034906

[2] Gonzalez, H.A., Huang, J., Kelber, F., Nazeer, K.K., Langer, T., Liu, C., Lohrmann, M., Rostami, A., Schöne, M., Vogginger, B., et al.: Spinnaker2: A large-scale neuromorphic system for event-based and asynchronous machine learning. arXiv preprint arXiv:2401.04491 (2024)

[3] Mayr, C., Hoeppner, S., Furber, S.: Spinnaker 2: A 10 million core processor system for brain simulation and machine learning. arXiv preprint arXiv:1911.02385 (2019)

[4] Vogginger, B., Kelber, F., Jobst, M., Yan, Y., Gerhards, P., Weih, M., Akl, M.: py-spinnaker2. Zenodo (2023). https://doi.org/10.5281/zenodo.10202110 . https:

//doi.org/10.5281/zenodo.10202110

[5] Neumarker, F., Höppner, S., Dixius, A., Mayr, C.: True random number generation from bang-bang adpll jitter. In: 2016 IEEE Nordic Circuits and Systems Conference (NORCAS), pp. 1–5 (2016). https://doi.org/10.1109/NORCHIP.2016.7792875

[6] Pierro, A., Stratmann, P., Guerra, G.A.F., Risbud, S., Shea, T., Mangalore, A.R., Wild, A.: Solving QUBO on the Loihi 2 Neuromorphic Processor (2024)

[7] Matsuda, Y.: Benchmarking the MAX-CUT problem on the Simulated Bifurcation Machine — medium.com. https://medium.com/toshiba-sbm/benchmarking-the-max-cut-problem-on-the-simulated-bifurcation-machine-e26e1127c0b0. [Accessed 20-04-2024]